CHUYÊN ĐỀ
SHORTEST PATH

(Đường đi ngắn nhất trong đồ thị)

---

1. Giới thiệu bài toán: Tìm Đường Đi Ngắn Nhất

1.1. Shortest Path là gì?

Giả sử bạn đang đứng ở nhà và muốn đi đến trường. Có rất nhiều con đường để đi: có đường đi thẳng, có đường phải vòng qua chợ, có đường lại phải đi qua nhà đứa bạn thân. Mục tiêu của chúng ta là tìm ra con đường tốn ít thời gian nhất, ít khoảng cách nhất, hoặc ít xăng nhất. Trong Tin học, người ta gọi đó là bài toán **Shortest Path** (Tìm đường đi ngắn nhất).

1.2. Ví dụ đời thực

- **Google Maps:** Khi bạn nhập điểm bắt đầu và điểm kết thúc, Google Maps sẽ tính toán và vẽ ra một đường màu xanh dương. Đó chính là đường đi ngắn nhất!
- **Trò chơi mê cung:** Tìm cách đi từ cửa vào đến cửa ra sao cho tốn ít bước chân nhất.
- **Giao hàng:** Các anh shipper cần tìm đường đi giao trà sữa cho bạn nhanh nhất để đá không bị tan.

---

2. Bản đồ của chúng ta: Biểu diễn Đồ thị

Để máy tính hiểu được "bản đồ", chúng ta cần biến nó thành **Đồ thị** (Graph). Đồ thị được tạo nên từ 2 thứ cực kỳ đơn giản:

2.1. Đỉnh (Vertex) và Cạnh (Edge)

- **Đỉnh:** Là các địa điểm. Ví dụ: Nhà bạn, Trường học, Quán trà sữa.
- **Cạnh:** Là con đường nối giữa 2 địa điểm đó.

2.2. Trọng số (Weight)

- **Đồ thị không trọng số:** Các con đường đều dài bằng nhau (ví dụ: mỗi bước đi trong mê cung tính là 1 bước).
- **Đồ thị có trọng số:** Các con đường dài ngắn khác nhau. Con đường dài 5km sẽ có "trọng số" là 5.

2.3. Hướng đi (Directed vs Undirected)

- **Vô hướng (Undirected):** Đường 2 chiều. Bạn đi từ Nhà đến Trường được thì cũng đi từ Trường về Nhà được.
- **Có hướng (Directed):** Đường 1 chiều. Chỉ được đi theo chiều mũi tên chỉ dẫn.

2.4. Cách dạy máy tính đọc bản đồ

Máy tính không có mắt để nhìn bản đồ như chúng ta, nên ta dùng 2 cách:

- **Ma trận kề (Adjacency Matrix):** Kẻ một cái bảng vuông. Hàng là điểm xuất phát, cột là điểm đến. Ô giao nhau ghi độ dài con đường. Cách này dễ hiểu nhưng rất tốn bộ nhớ!
- **Danh sách kề (Adjacency List):** Ghi ra một cuốn sổ. Trang 1 ghi "Từ nhà có thể đi đến: Trường (2km), Chợ (1km)". Cách này cực kỳ tiết kiệm bộ nhớ và được dùng nhiều nhất.

---

3. Thuật toán BFS: Vết Dầu Loang (Cho đồ thị không trọng số)

3.1. Ý tưởng

Hãy tưởng tượng bạn nhỏ một giọt dầu xuống mặt nước. Vết dầu sẽ loang ra xung quanh thành từng vòng tròn. Vòng thứ 1 cách tâm 1 bước, vòng thứ 2 cách tâm 2 bước... Thuật toán BFS (Breadth-First Search - Tìm kiếm theo chiều rộng) hoạt động y hệt như vậy!

3.2. Tại sao BFS lại tìm được đường ngắn nhất?

Vì nó lan tỏa đều ra mọi hướng! Nếu nó chạm tới "Trường học" ở vòng loang thứ 3, thì chắc chắn 3 bước là con đường ngắn nhất. Không thể có đường nào 2 bước mà đến được, vì nếu có thì vòng loang thứ 2 đã chạm tới trường rồi!

3.3. Bài toán dẫn nhập: Giải cứu thú cưng

Mô tả đề bài: Bản đồ khu phố là một đồ thị vô hướng, không trọng số (mỗi con đường nối 2 ngã tư đều dài 1 bước). Bạn đang ở đỉnh S. Thú cưng của bạn đang đi lạc ở đỉnh F. Hãy tìm số bước ít nhất để đi từ S đến F.

Input & Output:

Định dạng Input	Định dạng Output
- Dòng 1: Số đỉnh N, số cạnh M, đỉnh bắt đầu S, đỉnh đích F. - M dòng tiếp theo: Mỗi dòng chứa 2 số u, v thể hiện có đường nối giữa ngã tư u và ngã tư v.	Một số nguyên duy nhất là số bước ít nhất từ S đến F. Nếu không có đường đi, in ra -1.

Ví dụ minh họa:

Input	Output
4 4 1 4 1 2 2 3 3 4 1 3	2

Giải thích ví dụ: Bạn ở đỉnh 1, đích là đỉnh 4. Đường đi ngắn nhất là $1 \to 3 \to 4$ (tốn 2 bước).

3.4. Mô phỏng từng bước và Code

Chúng ta sẽ dùng một cái hàng đợi (Queue - giống như xếp hàng mua vé). Ai vào hàng trước sẽ được xử lý trước.

```cpp
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

const int MAXN = 100005;
vector<int> adj[MAXN]; // Danh sach ke luu ban do
int dist[MAXN];        // Luu khoang cach tu dinh bat dau

void bfs(int start) {
    // Buoc 1: Chuan bi
    for(int i = 0; i < MAXN; i++) dist[i] = -1; // -1 nghia la chua di toi
    queue<int> q;

    // Buoc 2: Cho diem bat dau vao hang doi
    q.push(start);
    dist[start] = 0; // Tu vi tri hien tai den chinh no la 0 buoc

    // Buoc 3: Lan toa (loang)
    while(!q.empty()) {
        int u = q.front(); // Lay nguoi dung dau hang ra
        q.pop();

        // Nhin ra xung quanh xem u noi voi ai
        for(int v : adj[u]) {
            if(dist[v] == -1) {     // Neu chua tung den v
                dist[v] = dist[u] + 1; // Khoang cach tang them 1 buoc
                q.push(v);          // Cho v vao hang de lan toa tiep
            }
        }
    }
}
```

Độ phức tạp: $O(V + E)$ (với V là số đỉnh, E là số cạnh). Chạy cực kỳ nhanh!

---

4. Thuật toán Dijkstra: Người Giao Hàng Cẩn Thận (Đồ thị có trọng số)

4.1. Khi nào dùng?

Vết dầu loang (BFS) chỉ đúng khi độ dài các con đường bằng nhau. Nếu từ Nhà đến Chợ là 1km, nhưng từ Nhà đến Trường là 10km, BFS sẽ sai bét! Lúc này, ta cần mời "chuyên gia" **Dijkstra** ra mặt.

4.2. Ý tưởng tham lam (Greedy)

Hãy tưởng tượng bạn là một nhà thám hiểm thông minh đang đi trong một khu rừng lớn. Mỗi con đường có độ dài khác nhau. Bạn không muốn đi lòng vòng, bạn luôn muốn tìm cách nhanh nhất để đến đích.

Dijkstra có một "luật chơi" rất đơn giản nhưng cực kỳ thông minh, gọi là **tham lam (Greedy)**:

> **Luật chơi của Dijkstra:**
> "Ở mỗi bước, hãy nhắm mắt chọn con đường đang ngắn nhất mà bạn đã biết, và tin chắc rằng con đường đó **không thể ngắn hơn được nữa**".

Nói cách khác: Mỗi lần bạn chọn một thành phố (đỉnh) mà bạn tin là đã tìm được khoảng cách ngắn nhất từ nhà đến thành phố đó. Sau đó bạn chỉ "lan tỏa" thông tin từ thành phố này sang các thành phố lân cận. Vì bạn đã chắc chắn đây là khoảng cách tốt nhất, nên bạn không cần quay lại xem xét nó lần nữa.

Tại sao cách làm "tham lam" này lại đúng?

Giả sử có hai con đường đi đến cùng một thành phố X: - Con đường A: dài 10km - Con đường B: dài 7km

Nếu bạn đã tìm được con đường B (7km), thì rõ ràng con đường A (10km) không thể tốt hơn. Vậy bạn không cần quan tâm đến con đường A nữa.

Dijkstra làm đúng như vậy: Nó luôn xử lý trước những đỉnh có khoảng cách nhỏ nhất trước. Khi một đỉnh được chọn ra để xử lý, nghĩa là khoảng cách đến nó lúc đó là ngắn nhất có thể, và sẽ không bao giờ có đường nào tốt hơn nữa.

Lưu ý quan trọng (rất dễ nhớ): Thuật toán Dijkstra chỉ đúng khi **tất cả trọng số cạnh đều không âm** (không có đường nào có độ dài âm). Nếu có đường âm, luật "tham lam" này sẽ sai, và chúng ta phải dùng thuật toán khác.

Hình dung đơn giản như trò chơi: Bạn có một danh sách các thành phố cần khám phá. Mỗi lần bạn luôn chọn thành phố "gần nhất mà bạn chắc chắn" để khám phá tiếp. Cứ làm như vậy cho đến khi khám phá hết tất cả. Đó chính là tinh thần Greedy của Dijkstra!

4.3. Tóm tắt thuật toán Dijkstra

Thuật toán Dijkstra hoạt động theo nguyên tắc "luôn chọn con đường hứa hẹn tốt nhất ngay lúc này". Dưới đây là các bước được viết cực kỳ đơn giản, dễ hình dung như đang chơi một trò chơi:

1. **Khởi tạo**: Đánh dấu khoảng cách từ đỉnh nguồn (điểm xuất phát) là 0. Tất cả các đỉnh còn lại đánh dấu là "vô cùng" (∞). Nghĩa là lúc này ta chưa biết đường đến chúng dài bao nhiêu.

2. **Chọn đỉnh tốt nhất hiện tại**: Trong số tất cả các đỉnh **chưa xét xong**, tìm đỉnh có giá trị đánh dấu nhỏ nhất. Gọi đỉnh này là A (current node - đỉnh hiện tại).

3. **Cập nhật cho các hàng xóm**: Xem tất cả các đỉnh kề (neighbour) của đỉnh A. Với mỗi đỉnh kề B, ta tính thử:

   $$\text{khoảng cách mới} = \text{khoảng cách của } A + \text{trọng số cạnh } A \to B$$

   Nếu khoảng cách mới này **nhỏ hơn** giá trị đang ghi ở đỉnh B, thì ta cập nhật lại giá trị cho B thành khoảng cách mới tốt hơn.

4. **Đánh dấu đã xong**: Sau khi đã cập nhật hết các hàng xóm của A, ta đánh dấu A là "đã xét xong". Từ nay ta sẽ không quay lại đỉnh A nữa.

5. **Lặp lại**: Nếu vẫn còn đỉnh nào chưa xét xong, quay lại bước 2. Ngược lại, thuật toán kết thúc.

Hình dung trực quan: Bạn đang cầm một tấm bản đồ có rất nhiều thành phố. Ban đầu chỉ biết nhà mình là 0km. Mỗi lần bạn chọn thành phố gần nhất mà bạn chắc chắn đã tìm được đường ngắn nhất đến nó, rồi "lan tỏa" thông tin khoảng cách tốt hơn đến các thành phố lân cận. Cứ làm như vậy cho đến khi mọi thành phố đều đã được "xác nhận" khoảng cách ngắn nhất.

Lưu ý: Thuật toán này chỉ đúng khi tất cả trọng số cạnh đều không âm (không có đường âm).

4.4. Mã giả (Pseudocode): Thuật toán Dijkstra

Trước khi viết code C++ thật, chúng ta hãy xem qua **mã giả** (pseudocode) để dễ hình dung cách thuật toán hoạt động.

Hãy tưởng tượng `dist[]` là cuốn sổ tay nhỏ của bạn, dùng để ghi khoảng cách ngắn nhất từ nhà đến từng nơi. Còn `Q` là danh sách những địa điểm bạn đang chờ khám phá.

```
// Pseudocode of Dijkstra's Algorithm

void Dijkstra(Graph, source):
    // Step 1: Initialize
    dist[source] = 0        // Distance from source to itself is 0
    for each vertex v in Graph:
        if v != source:
            dist[v] = infinity   // We don't know the way yet

    Q = all vertices in the graph     // Put every vertex into the "to explore" list

    // Step 2: Main exploration loop
    while Q is not empty:
        v = vertex in Q with the smallest dist[v]   // Choose the closest unprocessed vertex
        remove v from Q                             // We have finished processing v

        // Step 3: Check all neighbors
        for each neighbor u of v:
            // If going through v gives a shorter path to u
            if dist[u] > dist[v] + length(v, u):
                dist[u] = dist[v] + length(v, u)    // Update the notebook with better distance

    return dist[]                          // Now dist[] contains shortest paths from source
```

Tóm tắt bí kíp của Dijkstra bằng lời dễ nhớ:

Ở mỗi bước, bạn luôn làm hai việc đơn giản:

- Nhắm mắt chọn ra địa điểm **gần nhất** mà bạn chưa xử lý xong (`the vertex with min dist[]`).
- Từ địa điểm đó, nhìn sang tất cả các nhà hàng xóm và hỏi: "Nếu đi qua đây rồi mới đến nhà hàng xóm, có gần hơn đường cũ không?" Nếu có thì cập nhật ngay lập tức vào cuốn sổ tay (`dist[]`).

Cứ lặp lại như vậy cho đến khi danh sách chờ `Q` hết sạch. Khi đó, cuốn sổ tay `dist[]` của bạn sẽ ghi đúng khoảng cách ngắn nhất từ điểm xuất phát đến mọi nơi khác!

Hình dung vui: Bạn giống như một người đưa thư siêu thông minh. Mỗi lần bạn luôn chọn ngôi nhà gần nhất mà bạn chắc chắn đã tìm được đường ngắn nhất, rồi từ đó đi thông báo cho các ngôi nhà xung quanh xem có thể đi tắt qua nhà bạn để gần hơn không.

4.5. Bài toán dẫn nhập: Shipper siêu tốc

Mô tả đề bài: Thành phố có N ngã tư và M con đường có chiều dài (thời gian di chuyển) khác nhau. Một shipper xuất phát từ nhà hàng (đỉnh 1) và cần giao hàng đến nhà khách hàng (đỉnh N) thật nhanh nhất có thể.

Hãy giúp shipper tìm ra **thời gian ngắn nhất** để đi từ đỉnh 1 đến đỉnh N.

Input:

- Dòng đầu tiên: hai số nguyên N và M (N là số ngã tư, M là số con đường).
- M dòng tiếp theo: mỗi dòng chứa ba số nguyên u, v, w nghĩa là có một con đường một chiều từ ngã tư u đến ngã tư v mất thời gian w.

Output:

- Một số nguyên duy nhất là thời gian ngắn nhất để shipper đi từ nhà hàng (đỉnh 1) đến nhà khách hàng (đỉnh N).
- Nếu không có đường nào đi được, có thể in ra -1 (tùy theo yêu cầu bài toán, ở đây giả sử luôn có đường).

Ví dụ minh họa:

Input	Output
3 3 1 2 2 2 3 1 1 3 5	3

Giải thích ví dụ: - Đi thẳng từ $1 \to 3$ tốn 5 đơn vị thời gian. - Nhưng nếu đi vòng $1 \to 2 \to 3$ thì chỉ tốn $2 + 1 = 3$ đơn vị thời gian.

Dijkstra sẽ tự động tìm ra cách đi vòng ngắn hơn này!

Lưu ý: Trong bài toán thực tế, đồ thị có thể là vô hướng (đường hai chiều). Khi đó ta cần thêm cả hai chiều $u \to v$ và $v \to u$ vào đồ thị.

4.6. Code C++ với Priority Queue

Thay vì dùng hàng đợi thường như trong BFS, Dijkstra sử dụng **Hàng đợi ưu tiên (Priority Queue)**. Hàng đợi này có phép thuật: nó luôn tự động đưa ra đỉnh có khoảng cách ngắn nhất hiện tại!

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ii pair<long long, int>

const int MAXN = 100005;
const long long INF = 1e18;

vector<ii> adj[MAXN];
long long dist[MAXN];

void dijkstra(int start) {
    for(int i = 0; i < MAXN; i++) dist[i] = INF;

    priority_queue<ii, vector<ii>, greater<ii>> pq;

    dist[start] = 0;
    pq.push({0, start});

    while(!pq.empty()) {
        long long d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if(d > dist[u]) continue;

        for(auto edge : adj[u]) {
            int v = edge.first;
            long long w = edge.second;

            if(dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Độ phức tạp: $O((V + E) \log V)$.

Lưu ý quan trọng: Dijkstra chỉ hoạt động đúng khi tất cả trọng số cạnh đều không âm (≥ 0). Nếu có cạnh âm, thuật toán sẽ cho kết quả sai.

---

5. Thuật toán Floyd-Warshall: Điểm Trung Chuyển (Mọi cặp đỉnh)

5.1. Khi nào dùng?

Nếu như BFS và Dijkstra chỉ giúp chúng ta tìm đường ngắn nhất **từ một điểm xuất phát** (ví dụ: từ nhà bạn đến trường), thì Floyd-Warshall lại có siêu năng lực khác:

Nó có thể tính khoảng cách ngắn nhất giữa **tất cả các cặp đỉnh** trong đồ thị chỉ bằng **một lần chạy**!

Nghĩa là:

- Từ nhà bạn đến trường,
- Từ nhà bạn đến siêu thị,
- Từ trường đến công viên,
- Từ siêu thị đến nhà bạn bè...

Floyd-Warshall sẽ tính ra **hết tất cả** chỉ trong một lần.

Bạn có thể hình dung nó như một "bản đồ thông minh" biết trước khoảng cách giữa mọi đôi điểm trong thành phố, chứ không chỉ từ một điểm duy nhất.

Khi nào nên dùng Floyd-Warshall?

- Khi N (số đỉnh) khá nhỏ, thường là $N \leq 400$ (vì độ phức tạp là $O(N^3)$).
- Khi bài toán yêu cầu biết khoảng cách ngắn nhất giữa **mọi cặp đỉnh**, không chỉ từ một đỉnh nguồn.
- Khi đồ thị nhỏ nhưng cần thông tin đầy đủ về tất cả các đường đi.

Ngược lại, nếu N lớn (hàng nghìn đỉnh) thì Floyd-Warshall sẽ chạy quá chậm, lúc đó ta nên dùng Dijkstra nhiều lần hoặc các thuật toán tối ưu khác.

5.2. Ý tưởng Quy hoạch động (DP)

Floyd-Warshall dựa trên một ý tưởng rất thông minh của Quy hoạch động (Dynamic Programming).

Hãy tưởng tượng bạn đang cầm một bảng khoảng cách 'dist', trong đó 'dist[A][B]' nghĩa là khoảng cách ngắn nhất hiện biết từ điểm A đến điểm B.

Quy tắc vàng của Floyd-Warshall cực kỳ đơn giản:

> **"Từ điểm A đến điểm B, nếu mình đi vòng qua một điểm trung gian K thì có ngắn hơn không?"**

Với mọi cặp điểm A và B, ta kiểm tra và cập nhật theo công thức sau:

```
dist[A][B] = min(dist[A][B], dist[A][K] + dist[K][B]
```

Nghĩa là: Nếu đi đường `A → K → B` ngắn hơn đường hiện tại từ `A → B` , thì ta cập nhật lại `dist[A][B]` thành giá trị nhỏ hơn.

Ban đầu, bảng `dist` chỉ chứa khoảng cách của những cạnh nối trực tiếp (và `dist[i][i] = 0`). Nhưng mỗi khi thử thêm một điểm trung gian K (từ 1 đến N), ta dần dần cập nhật được những đường đi tốt hơn bằng cách "đi vòng qua K".

Hình dung vui: Giống như bạn đang xem một bảng điểm số giữa tất cả học sinh. Mỗi lần chọn một bạn K làm "cầu nối", bạn kiểm tra xem qua bạn K thì khoảng cách giữa hai bạn khác có gần hơn không. Cứ thử hết tất cả K, cuối cùng bảng của bạn sẽ chứa khoảng cách ngắn nhất giữa mọi cặp điểm.

Đó chính là sức mạnh của Floyd-Warshall – một cách làm Quy hoạch động rất đẹp và dễ hiểu!

4.5. Bài toán dẫn nhập: Shipper siêu tốc

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả đề bài: Thành phố có N ngã tư và M con đường một chiều có chiều dài (thời gian di chuyển) khác nhau. Shipper xuất phát từ nhà hàng (đỉnh 1) và cần giao hàng đến nhà khách hàng (đỉnh N) thật nhanh nhất có thể.

Hãy giúp shipper tìm ra **thời gian ngắn nhất** để đi từ đỉnh 1 đến đỉnh N.

Input

- Dòng đầu tiên: hai số nguyên N và M.
- M dòng tiếp theo: mỗi dòng chứa ba số nguyên u, v, w nghĩa là có một con đường một chiều từ ngã tư u đến ngã tư v mất thời gian w.

Output

- In ra một số nguyên duy nhất là thời gian ngắn nhất để shipper đi từ nhà hàng (đỉnh 1) đến nhà khách hàng (đỉnh N).
- Nếu không có đường nào đi được, in ra -1.

Điều kiện

- $1 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $1 \leq u, v \leq N$
- $0 \leq w \leq 10^9$

Ví dụ minh họa

Input	Output
3 3 1 2 2 2 3 1 1 3 5	3

Giải thích ví dụ:

- Từ thành phố 1 đến 3: đường ngắn nhất là $1 \to 2 \to 3$ với tổng khoảng cách $5 + 3 = 8$.
- Từ thành phố 1 đến 4: đường ngắn nhất là $1 \to 2 \to 3 \to 4$ với tổng khoảng cách $5 + 3 + 1 = 9$.

Floyd-Warshall sẽ tự động tính ra được cả hai kết quả này chỉ sau một lần chạy!

5.4. Code tham khảo C++

Khác với BFS và Dijkstra dùng danh sách kề, Floyd-Warshall sử dụng **ma trận kề** (ma trận khoảng cách). Thuật toán nổi tiếng với cấu trúc chỉ gồm **ba vòng lặp for lồng nhau**.

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

const int MAXN = 505;
const long long INF = 1e18;

long long dist[MAXN][MAXN];

void floydWarshall(int n) {
    for(int k = 1; k <= n; k++) {
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= n; j++) {
                if(dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

Độ phức tạp: $O(N^3)$

Lưu ý quan trọng: Thuật toán chạy khá chậm vì có ba vòng lặp lồng nhau. **Không nên dùng** khi số đỉnh $N > 400 \sim 500$, vì rất dễ bị Time Limit Exceeded (TLE).

---

6. Tổng kết: Bảng So Sánh

Sau khi học qua cả ba thuật toán, đây là bảng tổng hợp **"siêu dễ nhìn"** để bạn nhớ mãi không quên: **Khi nào nên dùng BFS, Dijkstra hay Floyd-Warshall?**

Đặc điểm	**BFS**	**Dijkstra**	**Floyd-Warshall**
Mục đích chính	Tìm đường ngắn nhất từ 1 đỉnh đến mọi đỉnh	Tìm đường ngắn nhất từ 1 đỉnh đến mọi đỉnh	Tìm đường ngắn nhất giữa **mọi cặp đỉnh**
Trọng số đồ thị	Chỉ dùng khi không có trọng số (tất cả cạnh bằng 1)	Dùng khi có trọng số và trọng số ≥ 0	Dùng được với mọi trọng số (không có chu trình âm)
Độ phức tạp	$O(V+E)$	$O((V+E)\log V)$	$O(V^3)$
Khuyên dùng khi	Mê cung, lưới, tìm ít bước nhất	Đồ thị lớn, đường đi có trọng số	Bản đồ nhỏ ($N \leq 400$), cần biết khoảng cách giữa tất cả các cặp đỉnh

Tóm tắt nhanh để chọn "vũ khí":

- **BFS** — Khi mọi con đường dài bằng nhau → Siêu nhanh và đơn giản nhất.
- **Dijkstra** — Khi đường có độ dài khác nhau và không có đường âm → Lựa chọn hàng đầu cho bản đồ thực tế.
- **Floyd-Warshall** — Khi N nhỏ và bạn muốn biết hết khoảng cách giữa mọi đôi điểm → Một lần chạy là xong tất!

7. Mở rộng: BFS và Dijkstra Đa Nguồn (Multi-source)

7.1. Dẫn nhập: Khi có quá nhiều vạch xuất phát!

Ở các phần trước, chúng ta đã quen với việc tìm đường đi từ **MỘT** điểm xuất phát (Nhà) đến các điểm khác. Nhưng thực tế thường "hành" chúng ta hơn thế:

- **Cứu hỏa:** Cả khu rừng đang cháy, nhưng thành phố lại có tận 3 trạm cứu hỏa. Ta cần biết thời gian ngắn nhất để **một xe cứu hỏa bất kỳ** (từ trạm gần nhất) chạy đến dập lửa ở từng ngôi nhà.
- **Lây lan:** Có 5 người cùng bị cảm cúm trong một lớp học. Hỏi sau bao lâu thì virus lây đến bạn ngồi bàn cuối?
- **Phát sóng:** Thành phố xây dựng 10 cột sóng wifi. Cần tính xem mỗi khu dân cư cách cột wifi gần nhất bao xa.

Cách "ngây thơ" nhất là: Có bao nhiêu nguồn thì ta chạy BFS/Dijkstra bấy nhiêu lần. Nếu có 1000 trạm cứu hỏa, ta chạy thuật toán 1000 lần → Máy tính sẽ khóc thét vì quá chậm (Time Limit Exceeded)!

Giải pháp ở đây cực kỳ ma thuật: **Thuật toán Đa nguồn (Multi-source)**.

7.2. BFS Đa Nguồn: Cơn Mưa Hạt Dầu

Ý tưởng trực quan cực dễ hiểu

Thay vì nhỏ **một giọt dầu** xuống mặt nước (BFS 1 nguồn), hãy tưởng tượng bạn cầm một nắm hạt dầu và ném **đồng loạt** xuống mặt hồ. Các vết dầu sẽ loang ra cùng lúc, đan xen vào nhau. Bất cứ điểm nào trên mặt hồ bị chạm tới đầu tiên bởi BẤT KỲ vòng loang nào, thì đó chính là khoảng cách ngắn nhất đến hạt dầu gần nhất!

Cách làm thực tế: Ở BFS cơ bản, ta cho 1 điểm xuất phát vào Queue (hàng đợi). Với BFS Đa nguồn, ta **nhét TẤT CẢ các điểm xuất phát vào Queue ngay từ lúc bắt đầu**, và gán khoảng cách của chúng bằng 0. Sau đó, cứ để BFS chạy bình thường như không có chuyện gì xảy ra!

Bài toán : Cứu hỏa rừng

Mô tả bài toán:
Khu rừng được chia thành một lưới ô vuông kích thước $N \times M$. Các ô trống ký hiệu là `'.'`, các ô đang cháy ký hiệu là `'F'`. Ngọn lửa cứ sau 1 phút sẽ lan sang các ô trống kề cạnh (trên, dưới, trái, phải). Hãy tính xem mất ít nhất bao nhiêu phút để lửa lan đến ô (x, y) bất kỳ.

Input

- Dòng đầu tiên gồm 2 số nguyên N, M.
- N dòng tiếp theo, mỗi dòng gồm M ký tự mô tả khu rừng (`'.'` hoặc `'F'`).

Output

- In ra lưới $N \times M$. Nếu ô đó là lửa ban đầu, in ra 0. Nếu ô đó bị cháy, in ra số phút ít nhất để lửa chạm tới. (Giả sử mọi ô đều sẽ bị cháy).

Ví dụ

Input	Output
3 4 . F . . . . F . F	1 0 1 2 2 1 2 1 1 0 1 0

Giải thích ví dụ:
Có 3 điểm cháy (F) ban đầu, chúng được tính là phút thứ 0. Các ô xung quanh lửa sẽ cháy ở phút thứ 1. Lửa lan từ nhiều phía cùng lúc, ô nào gặp lửa trước sẽ bị thiêu rụi ở phút đó.

Code C++ mô phỏng

```cpp
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

int n, m;
int dist[1005][1005];
string a[1005];

// 4 huong di chuyen: tren, phai, duoi, trai
```

```cpp
int dx[] = {-1, 0, 1, 0};
int dy[] = {0, 1, 0, -1};

void multiSourceBFS() {
    queue<pair<int, int>> q;

    // Buoc 1: Quet toan bo ban do, cho TAT CA cac diem chay vao Queue
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            if(a[i][j] == 'F') {
                dist[i][j] = 0;   // Lua co san thi mat 0 phut
                q.push({i, j});   // PUSH TAT CA vao queue
            } else {
                dist[i][j] = -1;  // Chua bi chay
            }
        }
    }

    // Buoc 2: Loang binh thuong
    while(!q.empty()) {
        int x = q.front().first;
        int y = q.front().second;
        q.pop();

        // Loang ra 4 huong
        for(int i = 0; i < 4; i++) {
            int nx = x + dx[i];
            int ny = y + dy[i];

            // Neu nam trong ban do va chua bi chay
            if(nx >= 0 && nx < n && ny >= 0 && ny < m && dist[nx][ny]
   == -1) {
                dist[nx][ny] = dist[x][y] + 1;
                q.push({nx, ny});
            }
        }
    }
}
```

7.3. Dijkstra Đa Nguồn: Cuộc Đua Của Các Shipper

Khi nào cần?

Giống hệt BFS Đa nguồn, nhưng lần này đường đi **có trọng số** (đường dài đường ngắn khác nhau).

Ý tưởng

Vẫn y xì đúc! Bạn nhớ "cuốn sổ tay" và "danh sách chờ Priority Queue" ở phần 4 chứ? Ban đầu, thay vì chỉ ghi Điểm Bắt Đầu vào sổ là 0 km, ta **ghi tất cả các điểm xuất phát vào sổ đều là 0 km**, và quăng hết chúng nó vào Priority Queue. Hàng đợi ưu tiên sẽ tự động làm nhiệm vụ của nó: đứa nào gần nhất tự khắc nổi lên đầu để đi mở đường tiếp!

Bài toán: Shipper Công Nghệ

Mô tả bài toán:
Thành phố có N ngã tư và M con đường có độ dài khác nhau. Có K anh shipper đang đứng sẵn ở K ngã tư khác nhau. Khi có một đơn hàng nổ ra ở ngã tư X bất kỳ, hãy tính xem khoảng cách ngắn nhất để **một anh shipper gần nhất** chạy tới X là bao nhiêu? (Cần tính cho tất cả N ngã tư).

Input

- Dòng 1: 3 số nguyên N, M, K.
- Dòng 2: K số nguyên là vị trí đứng ban đầu của các shipper.
- M dòng tiếp theo: 3 số u, v, w (Đường 2 chiều từ u đến v dài w).

Output

- In ra N số nguyên, số thứ i là khoảng cách từ shipper gần nhất đến ngã tư i.

Ví dụ

Input	Output
5 4 2 1 3 1 2 2 2 3 5 3 4 3 4 5 1	0 2 0 3 4

Giải thích ví dụ:
Shipper đứng ở 1 và 3 (khoảng cách bằng 0).
Điểm 2 cách 1 một đoạn dài 2, cách 3 một đoạn dài 5 → Chọn shipper ở điểm 1, khoảng cách là 2.
Điểm 4 cách 3 một đoạn dài 3 → Khoảng cách là 3.

Code C++ mô phỏng

```cpp
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

const long long INF = 1e18;
vector<pair<long long, int>> adj[100005];
long long dist[100005];
vector<int> shippers;

void multiSourceDijkstra(int n) {
    // Khoi tao vo cuc cho tat ca
    for(int i = 1; i <= n; i++) dist[i] = INF;

    priority_queue<pair<long long, int>,
                   vector<pair<long long, int>>,
                   greater<pair<long long, int>>> pq;
```

```
18
19      // Buoc 1: Cho TAT CA cac shipper vao PQ  v i  dist = 0
20      for(int start_node : shippers) {
21          dist[start_node] = 0;
22          pq.push({0, start_node});
23      }
24
25      // Buoc 2: Chay Dijkstra nhu binh thuong
26      while(!pq.empty()) {
27          long long d = pq.top().first;
28          int u = pq.top().second;
29          pq.pop();
30
31          if(d > dist[u]) continue;
32
33          for(auto edge : adj[u]) {
34              int v = edge.second;
35              long long w = edge.first;
36
37              if(dist[u] + w < dist[v]) {
38                  dist[v] = dist[u] + w;
39                  pq.push({dist[v], v});
40              }
41          }
42      }
43  }
```

7.4. So sánh nhanh: Nhận diện và Lựa chọn

Kỹ thuật Multi-source thực chất **chỉ là việc thay đổi bước Khởi tạo (Initialization)** của thuật toán. Phần vòng lặp cốt lõi bên dưới KHÔNG HỀ THAY ĐỔI.

Tiêu chí	BFS Đa nguồn	Dijkstra Đa nguồn
Dấu hiệu nhận biết	Hỏi khoảng cách ngắn nhất từ NHIỀU nguồn, trên đồ thị **không trọng số** (hoặc dạng bảng lưới/mê cung).	Hỏi khoảng cách ngắn nhất từ NHIỀU nguồn, trên đồ thị **có trọng số** khác nhau.
Cấu trúc dữ liệu	Queue bình thường (FIFO).	Priority Queue (Hàng đợi ưu tiên).
Bí kíp cốt lõi	Push tất cả đỉnh nguồn vào Queue ngay từ đầu.	Đặt `dist = 0` cho tất cả đỉnh nguồn và Push toàn bộ vào Priority Queue.

Lời khuyên: Bất cứ khi nào bạn đọc đề thấy có chữ "từ một trong các điểm...", "trạm gần nhất...", hãy nghĩ ngay đến việc quăng hết tụi nó vào chung một Hàng đợi nhé!

Bài tập vận dụng

Bài 1: Đường đi ngắn nhất

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một đồ thị vô hướng, có trọng số gồm n đỉnh và m cạnh. Tìm đường đi ngắn nhất từ đỉnh 1 đến đỉnh n.

Input

- Dòng đầu tiên gồm 2 số nguyên n và m.
- m dòng tiếp theo, mỗi dòng gồm 3 số nguyên u, v, w, thể hiện có cạnh trọng số w nối u và v.

Output

- In ra trọng số đường đi ngắn nhất trên một dòng, in ra đường đi trên một dòng tiếp theo.
- Hoặc in ra `-1` nếu không tồn tại đường đi. Trong trường hợp nhiều hơn 1 đường đi, in ra bất kì.

Điều kiện

- $1 \leq n, m \leq 2 \times 10^5$.
- $1 \leq u, v \leq n$.
- $1 \leq w \leq 10^9$.

Ví dụ

Input	Output
3 3 1 2 1 2 3 2 1 3 3	3 1 2 3

Giải thích ví dụ:

- Ta có thể đi trực tiếp từ $1 \to 3$ với độ dài là 3.
- Ta cũng có thể đi vòng $1 \to 2 \to 3$ với độ dài là $1 + 2 = 3$.
- Cả hai cách đều cho khoảng cách ngắn nhất là 3. Bạn có thể in ra một trong hai đường đi, ở ví dụ này chọn in ra $1 \to 2 \to 3$.

Bài 2: Giao hàng trạm trung chuyển

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một đồ thị vô hướng có trọng số gồm N đỉnh và M cạnh, biểu diễn mạng lưới giao thông. Một kiện hàng cần được vận chuyển từ đỉnh xuất phát S đến đỉnh đích T. Tuy nhiên, trên đường đi, kiện hàng bắt buộc phải ghé qua đỉnh trung chuyển X.

Hãy tìm tổng thời gian di chuyển ngắn nhất cho hành trình đi từ $S \to X \to T$.

Input

- Dòng 1: Gồm 5 số nguyên N, M, S, X, T.
- M dòng tiếp theo: Mỗi dòng gồm 3 số nguyên U, V, W thể hiện có một tuyến đường nối giữa U và V với thời gian di chuyển là W.

Output

- In ra tổng thời gian di chuyển ngắn nhất cho hành trình $S \to X \to T$.
- Nếu không tồn tại đường đi hợp lệ (không thể đến X từ S, hoặc không thể đến T từ X), in ra `-1`.

Điều kiện

- $1 \leq N \leq 10^3$.
- $1 \leq S, T, X, U, V \leq N$.
- $1 \leq M \leq \frac{N \times (N-1)}{2}$.
- $1 \leq W \leq 10^9$.

Ví dụ

Input	Output
4 4 1 3 4 1 2 2 2 3 1 3 4 2 1 4 1	5

Giải thích ví dụ:
Cần đi từ $1 \to 3 \to 4$.

- Chặng 1 ($1 \to 3$): Đường ngắn nhất là $1 \to 2 \to 3$ mất $2 + 1 = 3$.
- Chặng 2 ($3 \to 4$): Đi trực tiếp mất 2.
- Tổng thời gian: $3 + 2 = 5$.
- *Lưu ý:* Dù có đường đi trực tiếp $1 \to 4$ tốn 1, nhưng vì bắt buộc phải ghé qua 3 nên không được chọn.

Bài 3: Tốc độ cứu hộ (Đếm số lượng đường đi ngắn nhất)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Đội phòng cháy chữa cháy AS làm nhiệm vụ chữa cháy, hoả hoạn cho tất cả các địa điểm trong thành phố X. Với phương châm "Tốc độ cứu hộ, quyết định tất cả", họ phải xây dựng sẵn lộ trình đến các vị trí để khi cần là sẵn sàng lên đường. Khi có một đám cháy xảy ra tại một **khu vực nhất định** (D), xe cứu hỏa cần được cử đi từ **vị trí** S. Mỗi con đường giữa các khu vực/trạm cứu hỏa có một **thời gian di chuyển nhất định**.

Để đảm bảo hiệu quả chữa cháy tối đa, hãy:

- Xác định **thời gian di chuyển ngắn nhất** để xe cứu hỏa đến được khu vực cháy D.

- Đếm **số lượng các tuyến đường khác nhau** có cùng thời gian di chuyển ngắn nhất đó. Việc này cực kỳ quan trọng để đội cứu hỏa có các phương án dự phòng nhanh chóng nếu một tuyến đường chính bị tắc nghẽn hoặc không thể đi qua do sự cố bất ngờ.

Input

- Dòng 1: Bốn số nguyên N, M, S, D.
  - N: Tổng số địa điểm cần cứu hộ, được đánh số từ 1 đến N.
  - M: Số lượng tuyến đường kết nối các địa điểm.
  - S: Vị trí xuất phát.
  - D: Vị trí xảy ra đám cháy.
- M dòng tiếp theo: Mỗi dòng gồm ba số nguyên U, V, W.
  - U, V: Hai địa điểm được kết nối bởi một tuyến đường.
  - W: Thời gian di chuyển (trọng số) giữa U và V.

Output

- Dòng 1: In ra **thời gian di chuyển ngắn nhất** từ trạm cứu hỏa S đến khu vực cháy D.
- Dòng 2: In ra **số lượng các tuyến đường khác nhau** có cùng thời gian di chuyển ngắn nhất đó.
- Nếu không có đường đi từ S đến D, in ra `-1` trên cả hai dòng.

Ví dụ

Input	Output
6 8 1 6 1 2 5 1 3 10 2 4 10 3 4 5 3 5 15 4 5 5 4 6 10 5 6 5	25 4

Giải thích ví dụ:
Thời gian đi ngắn nhất từ điểm 1 đến điểm 6 là 25. Có 4 phương án di chuyển (tuyến đường) khác nhau cùng đạt được thời gian tối ưu này là:

- Lộ trình 1: $1 \to 2 \to 4 \to 6$ (Thời gian: $5 + 10 + 10 = 25$)
- Lộ trình 2: $1 \to 3 \to 4 \to 6$ (Thời gian: $10 + 5 + 10 = 25$)
- Lộ trình 3: $1 \to 3 \to 4 \to 5 \to 6$ (Thời gian: $10 + 5 + 5 + 5 = 25$)
- Lộ trình 4: $1 \to 2 \to 4 \to 5 \to 6$ (Thời gian: $5 + 10 + 5 + 5 = 25$)

Bài 4: Hệ thống chuyến bay tối ưu

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Bạn dự định đi từ thành phố Syrjälä (đỉnh 1) đến Lehmälä (đỉnh n) bằng máy bay. Mạng lưới gồm n thành phố và m chuyến bay một chiều. Hãy tìm câu trả lời cho 4 câu hỏi sau:

- Giá vé rẻ nhất của một lộ trình từ 1 đến n là bao nhiêu?
- Có bao nhiêu lộ trình đạt được giá rẻ nhất đó? (Kết quả lấy dư cho $10^9 + 7$).
- Số lượng chuyến bay tối thiểu của một lộ trình với giá rẻ nhất là bao nhiêu?
- Số lượng chuyến bay tối đa của một lộ trình với giá rẻ nhất là bao nhiêu?

Input

- Dòng đầu tiên gồm 2 số nguyên n và m: số lượng thành phố và chuyến bay.
- m dòng tiếp theo, mỗi dòng gồm 3 số nguyên a, b, c: có chuyến bay một chiều từ a đến b với giá vé c.
- Dữ liệu đảm bảo luôn có ít nhất một lộ trình từ 1 đến n.

Output

- In ra 4 số nguyên trên một dòng tương ứng với 4 câu hỏi trên.

Điều kiện

- $1 \leq n \leq 10^5$.
- $1 \leq m \leq 2 \cdot 10^5$.
- $1 \leq a, b \leq n$.
- $1 \leq c \leq 10^9$.

Ví dụ

Input	Output
4 5 1 4 5 1 2 4 2 4 5 1 3 2 3 4 3	5 2 1 2

Giải thích ví dụ:

Giá vé rẻ nhất từ 1 đến 4 là 5. Có 2 lộ trình đạt mức giá này:

- Lộ trình 1: $1 \to 4$ (Giá: 5, số chuyến bay: 1).
- Lộ trình 2: $1 \to 3 \to 4$ (Giá: $2 + 3 = 5$, số chuyến bay: 2).

Kết quả in ra: 5 (giá rẻ nhất), 2 (số lộ trình), 1 (số chuyến bay ít nhất), 2 (số chuyến bay nhiều nhất).

Bài 5: Robot leo núi (Hành trình trên cao nguyên)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Công ty Long Vân đang huấn luyện robot vận chuyển trên một địa hình lưới ô vuông kích thước $m \times n$. Ô tại dòng i, cột j có độ cao là h_{ij} và điểm thưởng là s_{ij}.

Quy tắc di chuyển của robot:

- Robot có thể bắt đầu tại **bất kỳ ô nào** trên lưới.
- Tại mỗi bước, robot có thể dừng lại hoặc di chuyển sang một ô chung cạnh có độ cao **lớn hơn hẳn** độ cao của ô hiện tại ($h_{mi} > h_c$).
- Mỗi khi đặt chân đến một ô, robot sẽ nhận được điểm thưởng tại ô đó.

Hãy tìm cách đặt robot và lộ trình di chuyển sao cho tổng điểm thưởng nhận được là lớn nhất.

Input

- Dòng đầu tiên gồm 2 số nguyên dương m, n.
- m dòng tiếp theo mô tả độ cao h_{ij}: Dòng thứ i chứa n số nguyên.
- m dòng tiếp theo mô tả điểm thưởng s_{ij}: Dòng thứ i chứa n số nguyên.

Output

- Một số nguyên duy nhất là tổng điểm lớn nhất mà robot có thể đạt được.

Điều kiện

- $1 \leq m \times n \leq 10^5$.
- $|h_{ij}| \leq 10^9$.
- $|s_{ij}| \leq 10^9$.

Ví dụ

Input	Output
3 4 1 2 3 4 4 5 5 7 8 7 6 5 1 1 1 1 1 1 1 1 1 1 1 1	7

Giải thích ví dụ:

Lộ trình tối ưu: Xuất phát từ ô $(1,1) \to (1,2) \to (1,3) \to (2,3) \to (3,3) \to (3,2) \to (3,1)$. Các ô đều có điểm thưởng là 1.

Bài 6: Xóa cạnh

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một đồ thị vô hướng, có trọng số gồm n đỉnh và m cạnh. Bạn cần tìm trọng số đường đi ngắn nhất từ đỉnh 1 đến đỉnh n.

Tuy nhiên, bạn có một khả năng đặc biệt: Bạn được phép chọn tối đa k cạnh bất kỳ trên hành trình của mình và gán trọng số của chúng bằng 0 (coi như đi miễn phí). Hãy tính toán chi phí tối ưu nhất sau khi đã sử dụng quyền trợ giúp này.

Input

- Dòng đầu tiên gồm 3 số nguyên n, m, k.
- m dòng tiếp theo, mỗi dòng gồm 3 số nguyên u, v, w thể hiện cạnh nối giữa u và v có trọng số w.

Output

- In ra trọng số đường đi ngắn nhất từ 1 đến n. Nếu không có đường đi, in ra `-1`.

Điều kiện

- $1 \leq n, m \leq 2 \times 10^5$.
- $1 \leq u, v \leq n$.
- $1 \leq w \leq 10^9$.
- $0 \leq k \leq 5$.

Ví dụ

Input	Output
9 6 2 7 9 4 5 2 5 3 7 7 7 6 3 8 6 8 8 1 7	7

Giải thích ví dụ:

Trong sơ đồ này, lộ trình từ đỉnh 1 đến đỉnh 9 là: $1 \to 8 \to 6 \to 7 \to 9$.

- Các cạnh trên đường đi có trọng số lần lượt là: $(1, 8) = 7, (8, 6) = 8, (6, 7) = 3, (7, 9) = 4$.
- Tổng chi phí ban đầu: $7 + 8 + 3 + 4 = 22$.
- Vì bạn có $k = 2$ quyền xóa cạnh, bạn sẽ chọn xóa 2 cạnh đắt nhất là $(8, 6)$ giá 8 và $(1, 8)$ giá 7 (hoặc cạnh có giá trị lớn tương đương nếu có).
- Chi phí còn lại: $3 + 4 = 7$. Đây chính là kết quả ngắn nhất tìm được.

Bài 7: Cạnh đôi (Dijkstra trên đồ thị trạng thái)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một đồ thị vô hướng có trọng số gồm n đỉnh và m cạnh. Quy tắc di chuyển của bạn rất đặc biệt: Bạn chỉ có thể di chuyển theo từng cặp 2 cạnh liên tiếp một lúc.

Cụ thể, nếu bạn đi từ đỉnh a sang b (cạnh w_{ab}), sau đó đi tiếp từ b sang c (cạnh w_{bc}), thì tổng chi phí cho "bước nhảy đôi" này là $w_{ab} \times w_{bc}$. Bạn không được phép dừng lại ở đỉnh b.

Hãy tìm chi phí nhỏ nhất để đi từ đỉnh 1 đến đỉnh n.

Input

- Dòng đầu tiên gồm 2 số nguyên n, m.
- m dòng tiếp theo, mỗi dòng gồm 3 số nguyên u, v, w thể hiện cạnh trọng số w nối giữa u và v.

Output

- In ra trọng số đường đi ngắn nhất từ 1 đến n. Nếu không thể đến được bằng các bước nhảy đôi, in ra `-1`.

Điều kiện

- $1 \leq n, m \leq 2 \times 10^5$.
- $1 \leq u, v \leq n$.
- $1 \leq w \leq 10$.

Ví dụ

Input	Output
3 3 1 2 1 2 3 2 1 3 3	2

Giải thích ví dụ:

- Lộ trình $1 \to 3$ trực tiếp không được tính vì bạn phải đi đúng 2 cạnh.
- Lộ trình $1 \to 2 \to 3$ là một bước nhảy đôi hợp lệ. Chi phí là $1 \times 2 = 2$. Đây là kết quả ngắn nhất.

Bài 8: Đường đi ngắn nhất thứ hai

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một đồ thị vô hướng liên thông gồm n đỉnh và m cạnh. Hãy tìm độ dài của con đường ngắn nhất thứ hai từ đỉnh 1 đến đỉnh n.

Lưu ý quan trọng: Độ dài của con đường này phải **lớn hơn hẳn** độ dài của con đường ngắn nhất.

Input

- Dòng đầu tiên gồm hai số nguyên n, m.
- m dòng tiếp theo, mỗi dòng gồm ba số nguyên u, v, w thể hiện có đường đi độ dài w giữa u và v.

Output

- In ra một số nguyên duy nhất là độ dài của con đường ngắn thứ hai tìm được.

Ràng buộc

- $1 \leq n, m \leq 10^5$.
- $1 \leq u, v \leq n$.
- $1 \leq w \leq 10^4$.

Ví dụ

Input	Output
4 4 1 2 100 2 4 200 2 3 250 3 4 100	450

Giải thích ví dụ:

- Đường ngắn nhất là: $1 \to 2 \to 4$ có tổng độ dài $100 + 200 = 300$.
- Đường ngắn thứ hai là: $1 \to 2 \to 3 \to 4$ có tổng độ dài $100 + 250 + 100 = 450$.
- Vì $450 > 300$ nên đây là kết quả cần tìm.

Bài 9: Tạm biệt cạnh lớn nhất

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một đồ thị vô hướng, có trọng số gồm n đỉnh và m cạnh. Thông thường, chi phí của một con đường là tổng trọng số của tất cả các cạnh. Nhưng ở bài toán này, bạn được hưởng một ưu đãi đặc biệt:

$$\text{Trọng số đường đi} = \sum w_i - \max(w_1, w_2, \ldots, w_k)$$

Nói cách khác, cạnh đắt nhất trên hành trình của bạn sẽ được miễn phí hoàn toàn. Hãy tìm trọng số đường đi nhỏ nhất từ đỉnh 1 đến đỉnh n.

Input

- Dòng đầu tiên gồm 2 số nguyên n, m.
- m dòng tiếp theo, mỗi dòng gồm 3 số nguyên u, v, w thể hiện có cạnh trọng số w nối giữa u và v.

Output

- In ra trọng số đường đi ngắn nhất từ 1 đến n sau khi đã trừ đi cạnh lớn nhất. Nếu không có đường đi, in ra `-1`.

Điều kiện

- $1 \leq n, m \leq 2 \times 10^5$.
- $1 \leq u, v \leq n$.
- $1 \leq w \leq 10^9$.

Ví dụ

Input	Output
3 3 1 2 1 2 3 2 1 3 3	0
10 6 8 7 2 2 4 10 3 9 4 1 5 3 7 5 5 5 7 9	-1

Giải thích ví dụ:

- Cách 1: Đi $1 \to 3$ trực tiếp. Chi phí: $3 - \max(3) = 0$.
- Cách 2: Đi $1 \to 2 \to 3$. Chi phí: $(1 + 2) - \max(1, 2) = 3 - 2 = 1$.
- Kết quả nhỏ nhất tìm được là 0.

Bài 10: Xây cầu tốc hành (Tối ưu hóa tổng khoảng cách)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một mạng lưới giao thông gồm n thành phố và m con đường. Để thúc đẩy kinh tế, nhà vua cho phép bạn xây dựng thêm **đúng một con đường thần kỳ** nối giữa hai thành phố bất kỳ với trọng số bằng 0 (đi lại không mất thời gian).

Mục tiêu của bạn là chọn vị trí xây dựng con đường này sao cho **tổng tất cả các quãng đường ngắn nhất giữa mọi cặp thành phố** là nhỏ nhất có thể. Nói cách khác, hãy cực tiểu hóa giá trị:

$$S = \sum_{i=1}^{n} \sum_{j=i+1}^{n} d(i, j)$$

Trong đó $d(i, j)$ là độ dài đường đi ngắn nhất giữa thành phố i và thành phố j.

Input

- Dòng đầu tiên gồm hai số nguyên n, m.

- m dòng tiếp theo, mỗi dòng gồm ba số nguyên u, v, w thể hiện con đường nối u và v có trọng số w.

Output

- In ra một số nguyên duy nhất là giá trị S nhỏ nhất có thể đạt được.

Giới hạn

- $1 \leq n \leq 100$.
- $1 \leq m \leq \frac{n(n-1)}{2}$.
- $1 \leq w \leq 10^4$.

Ví dụ

Input	Output
4 5 1 2 3 1 3 6 2 3 4 2 4 7 3 4 2	14

Giải thích ví dụ:
Nếu ta xây con đường thần kỳ nối giữa thành phố 1 và 4 với trọng số 0, tổng khoảng cách giữa các cặp thành phố sẽ giảm xuống mức tối thiểu là 14.

Bài 11: Quán trà sữa xóm Tà tưa (Multi-source Dijkstra biến tấu)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:
Ở khu xóm "Tà tưa" có n ngôi nhà và m con đường hai chiều. Có k ngôi nhà tự mở quán trà sữa. Người dân ở mỗi nhà đều muốn biết khoảng cách ngắn nhất để đi tới một quán trà sữa.

Tuy nhiên, có một điều thú vị:

- Nếu nhà bạn không có quán, hoặc có quán nhưng bạn thích uống đồ nhà làm ($t_i = 0$), khoảng cách là tới quán gần nhất (có thể là chính nhà mình).
- Nếu nhà bạn có quán nhưng bạn lại... "chê" đồ nhà làm ($t_i = 1$), bạn phải tìm đường đến một quán **khác** quán của mình.

Input

- Dòng đầu tiên gồm ba số nguyên n, m, k ($n, k \leq 10^5, m \leq 2 \times 10^5$).
- m dòng tiếp theo, mỗi dòng gồm ba số nguyên u, v, w ($1 \leq u, v \leq n, 1 \leq w \leq 10^9$) mô tả một con đường hai chiều nối giữa nhà u và nhà v với độ dài w.
- k dòng tiếp theo, mỗi dòng gồm hai số nguyên p_i và t_i ($1 \leq p_i \leq n, t_i \in \{0, 1\}$) mô tả quán trà sữa tại nhà p_i. Nếu $t_i = 1$, người ở nhà p_i muốn tìm quán khác; nếu $t_i = 0$, họ có thể uống tại nhà.

Output

- Một dòng duy nhất chứa n số nguyên $d_1, d_2, \ldots, d_n$, trong đó d_i là khoảng cách ngắn nhất từ ngôi nhà thứ i đến quán trà sữa phù hợp theo yêu cầu.

Ví dụ minh họa:

Input ví dụ	Output ví dụ
5 6 2 1 2 3 2 4 6 5 3 4 1 3 5 3 4 2 4 5 9 1 0 5 1	0 3 4 6 9

Giải thích ví dụ:
- Nhà 1 có quán và $t = 0$ nên tự uống tại chỗ, khoảng cách là 0.
- Nhà 5 có quán nhưng $t = 1$, phải tìm quán khác. Quán gần nhất là ở nhà 1 (đường đi 5-3-1 tốn $4 + 5 = 9$).

Phân tích: "Nhà vô địch" và "Nhà á quân khác gốc"

Thông thường, Multi-source Dijkstra chỉ quan tâm đến quán gần nhất. Nhưng ở đây, ta cần tìm quán gần nhất **không trùng ID gốc**.

Ý tưởng là tại mỗi ngôi nhà u, chúng ta sẽ lưu lại:

- **dist1[u], id1[u]:** Khoảng cách và ID của quán gần nhất chạm tới u.
- **dist2[u], id2[u]:** Khoảng cách và ID của quán gần nhất **có ID khác id1[u]** chạm tới u.

Bài 12: Đề xuất xây đường (HSG TPHCM 2022)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Mạng lưới giao thông thành phố gồm N nút và M đường một chiều nối các cặp nút. Với ý định muốn giảm được độ dài của đường đi ngắn nhất từ nút trọng yếu S đến nút T ($S \neq T$), một danh sách gồm K đường hai chiều được đề xuất để xem xét xây dựng.

Yêu cầu: Viết một chương trình để chọn ra **đúng một đường** trong danh sách đề xuất trên để xây dựng sao cho độ dài đường đi ngắn nhất từ S đến T là nhỏ nhất có thể.

Input

- Dòng đầu tiên chứa năm số nguyên dương N, M, K, S, T ($N \leq 10^4, M \leq 10^5, K < 300, S, T \leq N$).
- M dòng tiếp theo, mỗi dòng chứa ba số nguyên dương A_i, B_i, L_i ($L_i \leq 1000$): có đường một chiều từ A_i đến B_i với độ dài L_i.
- K dòng tiếp theo, mỗi dòng chứa ba số nguyên dương U_j, V_j, Q_j ($Q_j \leq 1000$): con đường hai chiều được đề xuất nối giữa hai nút U_j và V_j với độ dài Q_j.

Output

- In ra một số nguyên là độ dài nhỏ nhất có thể của đường đi ngắn nhất từ S đến T sau khi xây dựng xong một con đường từ danh sách đề xuất. Nếu không có đường đi, in ra `-1`.

Điều kiện

- $1 \leq N \leq 10^4$.
- $1 \leq M \leq 10^5$.
- $K < 300$.
- $1 \leq L_i, Q_j \leq 1000$.

Ví dụ

Input	Output
4 5 3 1 4 1 2 13 2 3 10 3 1 15 3 4 16 4 1 18 1 3 21 2 3 5 2 4 20	33

Giải thích ví dụ:
Ban đầu, đường ngắn nhất từ 1 đến 4 là $1 \to 2 \to 3 \to 4$ với độ dài $13 + 10 + 16 = 39$. Thử xây các đường đề xuất:

- Xây đường $1 \leftrightarrow 3$ (21): Đường ngắn nhất là $1 \to 3 \to 4$ độ dài $21 + 16 = 37$.
- Xây đường $2 \leftrightarrow 3$ (5): Đường ngắn nhất là $1 \to 2 \to 3 \to 4$ độ dài $13 + 5 + 16 = 34$.
- Xây đường $2 \leftrightarrow 4$ (20): Đường ngắn nhất là $1 \to 2 \to 4$ độ dài $13 + 20 = 33$.

Kết quả tốt nhất là 33.

Bài 13: Thành phố trung tâm (CENTRE - Phân tích nút thắt)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Đất nước Peace có N thành phố và M con đường hai chiều. Thành phố 1 và thành phố N là hai trung tâm kinh tế lớn nhất. Nhà vua muốn chọn thêm một thành phố K nữa để đầu tư làm trung tâm kinh tế thứ ba.

Tuy nhiên, có một điều kiện ngặt nghèo: Trong thời gian thành phố K được nâng cấp (tạm dừng mọi hoạt động thông thương), **khoảng cách ngắn nhất giữa thành phố 1 và thành phố N không được phép thay đổi**. Nếu khoảng cách này tăng lên, nền kinh tế sẽ bị đình trệ.

Yêu cầu: Hãy đếm và liệt kê các thành phố có thể chọn làm trung tâm kinh tế thứ ba (không tính thành phố 1 và N).

Input

- Dòng đầu tiên gồm hai số nguyên dương N và M.
- M dòng tiếp theo, mỗi dòng gồm 3 số nguyên x_i, y_i và d_i: con đường nối giữa x_i, y_i có độ dài d_i.

Output

- Dòng đầu tiên: Số lượng thành phố S thỏa mãn điều kiện.
- S dòng tiếp theo: Mỗi dòng ghi một số thứ tự của thành phố được chọn (in ra theo thứ tự tăng dần).

Giới hạn

- $2 \leq N \leq 30,000$.
- $1 \leq M \leq 100,000$.
- $1 \leq d_i \leq 1,000$.

Ví dụ

Input	Output
6 6 1 2 1 2 3 1 3 6 1 1 4 100 4 5 100 5 6 100	2 4 5

Phân tích và Gợi ý: Kỹ thuật đếm đường đi ngắn nhất

Để giải quyết bài toán này, chúng ta cần xác định xem một thành phố i có phải là "điểm yếu chí mạng" (nút thắt cổ chai) trên mọi con đường ngắn nhất từ $1 \to N$ hay không. Nếu không phải, thành phố đó có thể được chọn làm trung tâm kinh tế thứ ba.

Ta gọi các mảng sau (được tính bằng 2 lần chạy Dijkstra từ 1 và N):

- `d1[i], cnt1[i]`: Khoảng cách ngắn nhất và số lượng đường đi ngắn nhất từ 1 đến i.
- `dN[i], cntN[i]`: Khoảng cách ngắn nhất và số lượng đường đi ngắn nhất từ N đến i.

1. Điều kiện để đỉnh i nằm trên đường đi ngắn nhất: Một đỉnh i nằm trên ít nhất một con đường ngắn nhất từ $1 \to N$ khi và chỉ khi:

```
d1[i] + dN[i] = d1[N] = dN[1]
```

(Nghĩa là tổng quãng đường đi vòng qua i không dài hơn quãng đường đi trực tiếp tối ưu).

2. Điều kiện để đỉnh i là "nút thắt" bắt buộc: Đỉnh i được gọi là "nút thắt" nếu **tất cả** các con đường ngắn nhất từ $1 \to N$ đều buộc phải đi qua nó. Điều này xảy ra khi:

- Đỉnh i nằm trên đường đi ngắn nhất (thỏa mãn điều kiện ở mục 1).
- Số lượng đường đi ngắn nhất đi qua i đúng bằng tổng số đường đi ngắn nhất của toàn đồ thị:

```
cnt1[i] x cntN[i] = cnt1[N] = cntN[1]
```

3. Kết luận bài toán: Thành phố i ($i \neq 1, i \neq N$) có thể được chọn làm trung tâm kinh tế thứ ba nếu nó **không phải** là nút thắt bắt buộc. Điều này xảy ra trong hai trường hợp:

1. Đỉnh i hoàn toàn không nằm trên bất kỳ đường đi ngắn nhất nào (`d1[i] + dN[i] > d1[N]`).

2. Đỉnh i có nằm trên đường đi ngắn nhất, nhưng vẫn còn những con đường ngắn nhất khác "né" được đỉnh này (`cnt1[i] x cntN[i] < cnt1[N]`).

Bài 14: ZIGZAG (Hành trình thu thập)

Thời gian tối đa: 1.0s
Bộ nhớ tối đa: 256MB

Mô tả bài toán:

Cho một lưới ô vuông kích thước $m \times n$. Mỗi ô trong bảng chứa một chữ số từ 0 đến 9. Có k ngôi nhà nằm tại các tọa độ (x_i, y_i) với $1 \leq i \leq k$.

Với mỗi ngôi nhà, ta cần tìm độ dài đường đi ngắn nhất từ một ô bất kỳ trên bảng đến ngôi nhà đó, sao cho trên hành trình di chuyển, robot phải đi qua các ô chứa đầy đủ các giá trị từ 0 đến 9 ít nhất một lần (không cần theo đúng thứ tự). Nếu không có đường đi thỏa mãn, in ra `-1`.

Chú ý:

- Tại mỗi ô, chỉ được phép di chuyển đến các ô chung cạnh với ô đang đứng.
- Mỗi ô có thể được đi qua nhiều lần.
- Độ dài đường đi được tính bằng số lượng ô đã đi qua trên hành trình đó (bao gồm cả ô xuất phát và ô kết thúc).

Input

- Dòng đầu tiên ghi 3 số nguyên dương m, n, k.
- m dòng tiếp theo, mỗi dòng chứa n chữ số mô tả giá trị của các ô trong bảng.
- k dòng tiếp theo, mỗi dòng chứa hai số nguyên x_i, y_i là tọa độ ngôi nhà thứ i.
- Các hàng được đánh số từ 1 (từ trên xuống dưới), các cột được đánh số từ 1 (từ trái sang phải).

Output

- In ra k dòng, dòng thứ i là độ dài đường đi ngắn nhất tìm được đến ngôi nhà thứ i. Nếu không có đường đi thỏa mãn, in ra `-1`.

Giới hạn

- **Subtask 1 (50% số điểm):** $m, n \leq 100, k = 1$.
- **Subtask 2 (50% số điểm):** $m, n \leq 100, k \leq m \times n$.

Ví dụ

Input	Output
3 4 1 0 1 2 3 4 5 6 7 8 9 0 1 1 1	10
1 10 1 0 1 2 3 4 5 6 7 8 9 1 4	13